

⚡ Realtime-VLA **FLASH**: Speculative Inference Framework for Diffusion-based VLAs

Jiahui Niu^{1,2*} Kefan Gu^{3,4} Yucheng Zhao^{4†} Shengwen Liang^{1‡}
Tiancai Wang^{4‡} Xing Hu¹ Ying Wang¹ Huawei Li¹

¹ State Key Lab of Processors, Institute of Computing Technology, CAS

² University of Chinese Academy of Sciences

³ Nanjing University ⁴ Dexmal

Page: <https://dexmal.github.io/realtime-vla-flash>

Abstract

Diffusion-based vision-language-action models (dVLAs) are promising for embodied intelligence but are fundamentally limited in real-time deployment by the high latency of full inference. We propose **Realtime-VLA FLASH**, a speculative inference framework that eliminates most full inference calls during replanning by introducing a lightweight draft model with parallel verification via the main model’s Action Expert and a phase-aware fallback mechanism that reverts to the full inference pipeline when needed. This design enables low-latency, high-frequency replanning without sacrificing reliability. Experiments show that on LIBERO, FLASH largely preserves task performance by replacing many 58.0 ms full-inference rounds with speculative rounds as fast as **7.8 ms**, lowering task-level average inference latency to 19.1 ms (**3.04×** speedup). We additionally demonstrate effectiveness on real-world conveyor-belt sorting, highlighting its practical impact for latency-critical embodied tasks.

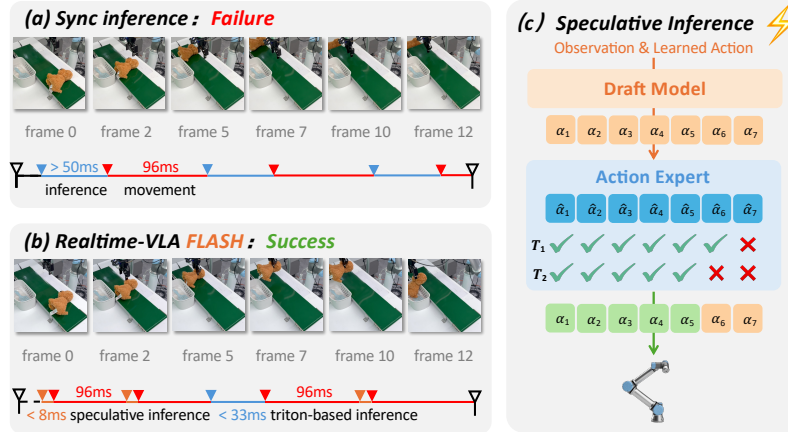


Figure 1: **Overview of Realtime-VLA FLASH.** (a) Standard synchronous dVLA inference runs the full pipeline at each replanning round, causing stale action updates and failure in latency-critical manipulation. (b) Realtime-VLA FLASH accelerates replanning, allowing the robot to react in time and complete the grasp. (c) FLASH introduces speculative inference for dVLAs: a lightweight draft model proposes a continuous action chunk, and the main model’s Action Expert reconstructs action endpoints from selected flow-matching timesteps in parallel to check consistency with the draft. The robot executes the longest consistent prefix and falls back to the full path when verification fails.

*This work was done during the internship at Dexmal.

†Project lead.

‡Corresponding author.

1 Introduction

Diffusion-based vision-language-action models (dVLAs) [1, 10, 20, 21] have recently emerged as an important paradigm for embodied intelligence. Their diffusion-based continuous action generation captures multimodal action distributions, making them well suited to complex manipulation tasks.

However, deploying dVLAs on real robotic systems remains challenging. In practice, robot control typically runs at a much higher frequency than model inference, as shown in Figure 1, and existing systems therefore rely on open-loop action chunking [33] to bridge this mismatch. For example, π_0 [1] predicts a chunk of future actions and replans only after several control steps have already been executed. While this design reduces the frequency of replanning, each replanning round still requires the same expensive full inference pipeline. As a result, end-to-end inference latency remains the main bottleneck, limiting the applicability of dVLAs to reactive, latency-sensitive tasks.

Speculative Inference aims to reduce repeated inference cost and has proven effective in large language models [12–14] and autoregressive VLAs [26, 34, 35]. Its core idea is to have a lightweight draft model propose candidates that are verified in parallel by the main model, thereby reducing expensive full generations. However, extending this paradigm to dVLAs is non-trivial. Speculative inference requires (1) a draft proposal, (2) parallel verification, and (3) an acceptance criterion, which are all well-defined for autoregressive models through token-level probabilities. dVLAs instead produce actions through iterative denoising in continuous space [5, 16], which challenges each ingredient: it is unclear what constitutes a meaningful draft through multi-step denoising, the sequential nature of denoising resists parallel verification, and the absence of explicit likelihoods leaves no natural acceptance criterion. The central bottleneck is therefore verification: *without a way to cheaply check a drafted continuous-action chunk, speculative inference collapses back to full sequential denoising.*

A key observation is that flow matching provides exactly such a structure. During flow-matching training, interpolation timesteps are sampled along linear paths between Gaussian noise and target actions, learning local flow constraints at intermediate states. Given a drafted action chunk, we can interpolate it with Gaussian noise, evaluate a few intermediate states in parallel using the Action Expert, and check whether the reconstructed endpoints agree with the draft, providing a cheap consistency check without full sequential denoising. In addition, consistency verification does not make speculative inference uniformly safe throughout the trajectory. For much of task execution, the robot performs smooth motions, where observations change slowly and small draft errors can often be tolerated. In fine-adjustment phases such as gripper switching, these errors can quickly accumulate and be amplified, leading to task failure. This motivates falling back to the full inference pipeline when verification or the task phase indicates that higher-fidelity actions are required.

In this paper, we introduce **Realtime-VLA FLASH**, a speculative inference framework for dVLAs. FLASH introduces a fast speculative path that uses a lightweight draft model to quickly propose a candidate action chunk, and then verifies its consistency with the main model’s Action Expert in parallel at a few selected timesteps, producing a dynamically sized executable action prefix. If no action prefix is accepted, FLASH falls back to the full path to correct the trajectory. FLASH also uses gripper switches as heuristic signals of upcoming fine-adjustment phases, where higher-quality full-path actions help prevent draft errors from accumulating into task failures. Thus, FLASH reduces average inference latency and improves action throughput while largely preserving task success.

We evaluate Realtime-VLA FLASH in simulation and real-world tasks. On LIBERO, with Triton optimization, FLASH runs speculative rounds as fast as **7.8 ms**, compared with 58.0 ms for full-inference rounds, lowering the task-level average latency to 19.1 ms (**3.04×** speedup) and improving action throughput by **2.63×**, while largely preserving task performance. On real conveyor-belt sorting, FLASH enables successful grasping at belt speeds up to 15 m/min, where other methods fail, demonstrating that speculative inference reduces effective inference latency and enables higher-speed, latency-sensitive manipulation tasks.

In summary, we make three contributions. First, we formulate speculative inference for flow-matching dVLAs and address its key challenge, continuous-action verification, using flow-matching interpolation paths. Second, we introduce Realtime-VLA FLASH, a dual-path inference runtime that combines a lightweight draft model, flow-consistency-based parallel verification, and phase-aware fallback to reduce expensive full-path inference during replanning. Third, we demonstrate on LIBERO and real-world tasks that FLASH replaces many expensive full-path rounds with low-cost speculative rounds, substantially reducing latency while largely preserving task success.

2 Related Work

2.1 Efficient Diffusion-based VLAs

Existing work has explored several ways to improve the efficiency of dVLAs [1, 10, 20, 21], among which the following two directions are most closely related to our setting.

Inference Pipeline Acceleration. Existing work on efficient dVLAs primarily reduces the cost of the original full inference pipeline through smaller models [28, 22, 15], layer compression [31], token pruning [25], quantization [28, 29, 32], and kernel/system-level optimization [19, 30]. These approaches improve the efficiency of the original inference pipeline, either by reducing the cost of particular components or by improving end-to-end execution efficiency. Our work is complementary to this line of research, but it addresses a different question at the control-loop level: whether every replanning round needs to invoke the full path.

Diffusion and Flow-Matching Acceleration. Another line of work accelerates diffusion- or flow-based policies [5, 16] by shortening the generation process itself, for example through distillation into few-step models [18, 27, 7] or direct one-step model training [8, 4]. These approaches reduce the cost of sequential denoising more directly, but they typically require additional training or modified policy formulations. In contrast, we keep the original flow-matching formulation intact and improve efficiency by exploiting otherwise idle compute during the Action Denoise stage.

2.2 Speculative Inference

Speculative Inference accelerates autoregressive generation by letting a lightweight draft model propose candidate outputs and a larger model verify them in parallel, thereby reducing expensive decoding steps. It has been widely studied in large language models [12–14] and extended to autoregressive VLAs [26, 34, 35], where discrete action tokens admit token-level probabilities that naturally support verification and acceptance.

Related acceleration has also been explored for stochastic diffusion models, where drafted denoising states can be checked using the transition structure of the stochastic reverse process [6, 9]. However, π_0 [1]-style dVLAs differ from both settings: they generate continuous action chunks under flow matching by integrating an ODE-defined velocity field, not discrete tokens or samples from a stochastic reverse kernel. Consequently, neither token-level verification nor transition-based diffusion verification applies directly, and the absence of explicit likelihoods leaves no natural acceptance criterion for drafted continuous-action chunks.

3 Methods

In this section, we present the architecture of Realtime-VLA FLASH. Section 3.1 revisits the dVLA inference pipeline and the latency bottlenecks revealed by profiling; Section 3.2 then summarizes the overall two-path design, and Sections 3.3–3.5 present the three core components of the flash path, namely draft action generation, multi-step parallel verification, and phase-aware fallback.

3.1 dVLA Preliminaries and Latency Analysis

dVLAs, such as π_0 [1], typically combine a pre-trained vision-language model (VLM) backbone [23, 24] with an Action Expert (AE). Perceptual context is passed from the VLM to the AE through the KV Cache. Formally, the model takes a multimodal observation

$$o_t = [I_t^1, \dots, I_t^n, \ell_t, q_t],$$

where I_t^i denotes the images, ℓ_t the language prompt, and q_t the robot state, and predicts a future action chunk

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H-1}].$$

Each per-step action a_t contains semantically distinct channels corresponding to end-effector position Δp_t , rotation $\Delta \theta_t$, and binary gripper control g_t . Under chunked execution, only a prefix of A_t is executed before the next action chunk is regenerated from the updated observation; we refer to each such refresh as a replanning round.

At inference time, the input images are first encoded into visual features, then concatenated with the language features, and the resulting multimodal prefix is processed by the VLM to construct a prefix KV Cache. This cache is then passed to the Action Expert, which predicts the action chunk by solving an observation-conditioned ODE from Gaussian noise:

$$\frac{dA_t^\tau}{d\tau} = v_\theta(A_t^\tau, \tau, o_t), \quad A_t^0 \sim \mathcal{N}(0, I),$$

where A_t^τ denotes the intermediate noisy action state at denoising time τ . At each denoising step, the Action Expert predicts the instantaneous denoising velocity field $v_\theta(A_t^\tau, \tau, o_t)$, and the final action chunk is obtained by numerically integrating this learned field over multiple timesteps, typically 10 denoising steps in the default setting, until $\tau = 1$. Consequently, a full inference round in π_0 -style dVLAs consists of three stages: Image Encoder, VLM prefill to construct the visual KV Cache, and a multi-step Action Denoise stage under the learned flow field.

We further profile these three stages on an NVIDIA RTX 4090D [11] and summarize the results with a roofline analysis in Figure 2. The measured latencies of Image Encoder, VLM prefill, and Action Denoise are 11.3 ms, 26.7 ms, and 20.0 ms, respectively. The roofline places Image Encoder and VLM prefill primarily in the compute-bound regime, suggesting that their latency should be reduced by eliminating redundant computation. For VLM prefill, this naturally motivates reusing the previously computed prefix KV Cache. In contrast, Action Denoise is memory-bound, since each denoising step repeatedly reads the cache while remaining unable to parallelize across steps. This leaves available compute resources underutilized and motivates parallel verification instead of repeatedly invoking the full path.

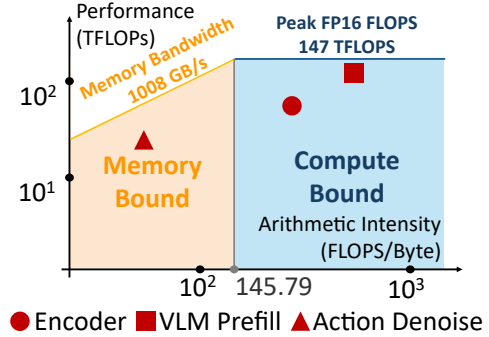


Figure 2: **Roofline analysis** of π_0 [1] inference on NVIDIA RTX 4090D. 145.79 is the balanced arithmetic intensity point.

3.2 Realtime-VLA FLASH Overview

Motivated by Section 3.1, Realtime-VLA FLASH adopts the dual-path architecture shown in Figure 3(a). In a flash-path round, FLASH still encodes the current images and feeds the features to a lightweight draft model, which proposes a candidate action chunk. The main model’s Action Expert then reuses the visual KV Cache produced by the most recent full-path round to reconstruct action endpoints at selected denoising timesteps and check their consistency. If no drafted action prefix

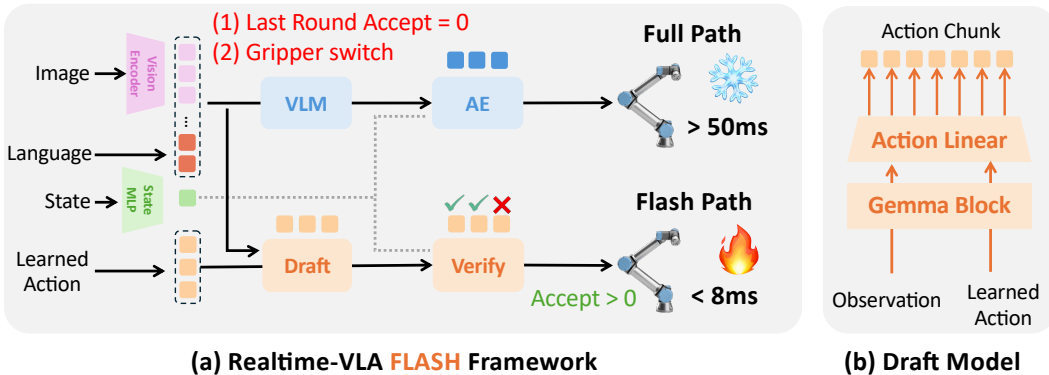


Figure 3: **Realtime-VLA FLASH Framework.** (a) Realtime-VLA FLASH uses two inference paths: the original full path (Full Path) and a lightweight speculative path (Flash Path). The Full Path performs Image Encoder, VLM prefill, and Action Denoise, while the flash path still encodes the current images but skips VLM prefill to draft a candidate action chunk for verification. (b) Draft model architecture. The draft model uses a single Gemma block with the same structure as the VLM, augments the input with learned action queries, and predicts the full action chunk in parallel through a linear action head.

is accepted, the cached context may no longer provide a reliable basis for speculative execution; FLASH therefore returns to the full path to refresh the context and correct the trajectory. In addition, FLASH uses gripper switches as phase-transition signals and proactively falls back to the full path during fine-adjustment phases, where higher-fidelity actions help mitigate error amplification.

3.3 Draft Action Generation

For flash-path rounds, candidate generation must be cheaper than invoking the full path. We therefore design a lightweight draft model that proposes a candidate action chunk for subsequent verification.

The draft model architecture in Figure 3(b) consists of a single Gemma block followed by a linear action head. At inference time, the flash path still runs the image encoder of the main model on the current images and combines the resulting visual features with the language instruction, while the robot state is projected into the same hidden space through a linear layer. We then append H learned action queries to the input sequence, one for each future timestep in the chunk. The attention mask follows the blockwise structure of the base model: visual and language tokens form the conditional prefix block, the state token forms a separate block, and the action queries form the action block. The action block attends to the full conditional prefix, allowing the model to predict the entire candidate chunk in parallel. The draft model is trained by supervised regression to the target action chunk,

$$\mathcal{L}_{\text{draft}} = \sum_{h=1}^H w_h \ell(\hat{a}_{t+h-1}^d, a_{t+h-1}),$$

where w_h is a step-dependent weight and $\ell(\cdot, \cdot)$ is the per-step action regression loss.

This design has several advantages. The draft model is lightweight, containing about 110M parameters in our implementation, compared with roughly 2.7B parameters in the VLM, which makes candidate generation substantially cheaper than invoking the full path. It also remains architecturally close to the main VLM and preserves the same action-chunk representation, making it easy to integrate and allowing the verification stage to reuse the Action Expert. Additional architectural and training details are provided in Appendix A.

3.4 Multi-step Parallel Verification

The remaining challenge is to verify a draft action chunk without rerunning the full sequential denoising process in Figure 4(a). In flow matching, this becomes possible because the policy learns the velocity field along the linear path between Gaussian noise and the endpoint at training time.

For a draft endpoint $\hat{A}_t^{(d)}$ and Gaussian noise ϵ , we construct an intermediate denoising state as

$$\tilde{A}_\tau = \tau \hat{A}_t^{(d)} + (1 - \tau)\epsilon,$$

where τ denotes denoising progress from Gaussian noise to the action endpoint. Under this convention, the Action Expert predicts the local velocity from the intermediate state toward the endpoint. Since the remaining denoising interval is $1 - \tau$, we reconstruct the endpoint by

$$\hat{A}_t(\tau) = \tilde{A}_\tau + (1 - \tau) v_\theta(\tilde{A}_\tau, \tau \mid c_t, s_t).$$

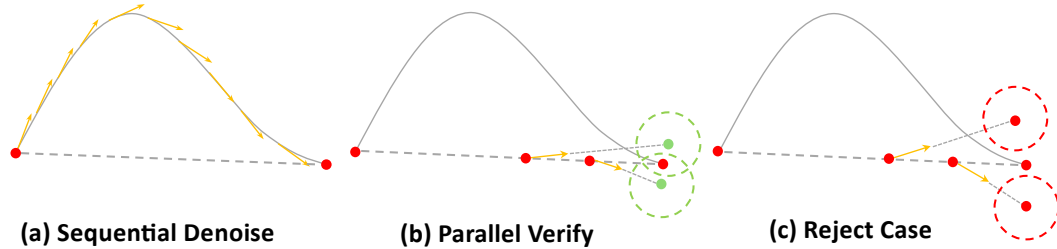


Figure 4: **Parallel verification.** (a) π_0 [1] generates an action chunk with flow matching through 10-step sequential denoising. (b) Realtime-VLA FLASH reconstructs endpoints from selected denoising timesteps in parallel and checks an action-by-action distance threshold within the chunk, yielding the accepted prefix. (c) If reconstructed endpoints deviate beyond the threshold, no prefix is accepted and the flash path falls back to the full path.

Algorithm 1 Multi-Step Parallel Verification

Require: Draft action chunk $\hat{A}_t^{(d)}$, reused visual KV Cache c_t , latest robot state s_t , verification timesteps $\mathcal{T} = \{\tau_1, \dots, \tau_K\}$, threshold δ

Ensure: Executable prefix length L

- 1: Sample a shared Gaussian noise ϵ for all verification timesteps
 - 2: **for all** $\tau_k \in \mathcal{T}$ **in parallel do**
 - 3: Construct intermediate state: $\tilde{A}_{\tau_k}^{(k)} = \tau_k \hat{A}_t^{(d)} + (1 - \tau_k) \epsilon$
 - 4: Predict local denoising velocity with the Action Expert: $v^{(k)} = v_\theta(\tilde{A}_{\tau_k}^{(k)}, \tau_k \mid c_t, s_t)$,
 - 5: Reconstruct endpoint estimate: $\hat{A}_t^{(k)} = \tilde{A}_{\tau_k}^{(k)} + (1 - \tau_k) v^{(k)}$
 - 6: **for** $h = 1, \dots, H$ **do**
 - 7: Compute step distance: $d_h^{(k)} = \text{Dist}_{\text{cont}}(\hat{a}_{t+h-1}^{(d)}, \hat{a}_{t+h-1}^{(k)})$
 - 8: **end for**
 - 9: Compute prefix length: $L^{(k)} = \sum_{h=1}^H \prod_{j=1}^h \mathbf{1}[d_j^{(k)} \leq \delta]$
 - 10: **end for**
 - 11: Return the conservative prefix length: $L = \min_{k=1, \dots, K} L^{(k)}$
-

If the draft is consistent with the main policy under the reused KV Cache c_t and the latest robot state s_t , these reconstructed endpoints should remain close to the draft across verification timesteps.

We therefore evaluate a small set of verification timesteps $\mathcal{T} = \{\tau_1, \dots, \tau_K\}$ in parallel and define the executable prefix as the longest leading segment whose included actions all satisfy the distance threshold at every verification timestep, as summarized in Algorithm 1 and Figure 4(b). We apply this distance-based rule to the continuous position and rotation channels through $\text{Dist}_{\text{cont}}$, while the gripper channel is treated separately because it encodes discrete semantic states rather than incremental motion. If $L = 0$, no executable prefix survives all verification checks, so the draft is rejected and control immediately returns to the full path, corresponding to the rejection case in Figure 4(c). Compared with binary acceptance, this rule returns an executable prefix, enabling more fine-grained execution and a smoother handoff to fallback. See Appendix B for further discussion.

3.5 Phase-aware Fallback

Verification in Section 3.4 is local: it checks the current draft under the reused context but does not anticipate transitions into fine-adjustment phases. Figure 5 compares the LIBERO-Spatial bowl-

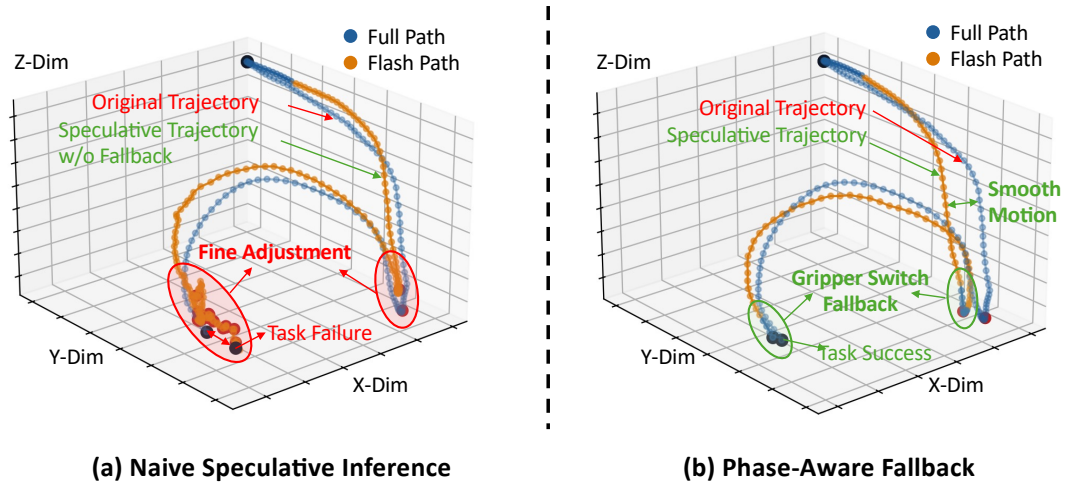


Figure 5: **Phase-aware fallback on a LIBERO-Spatial task trajectory.** 3D trajectories for a bowl-to-plate task. (a) Without phase-aware fallback, the flash path drifts during final placement and fails near the plate edge. (b) With fallback, final placement returns to the full path and succeeds.

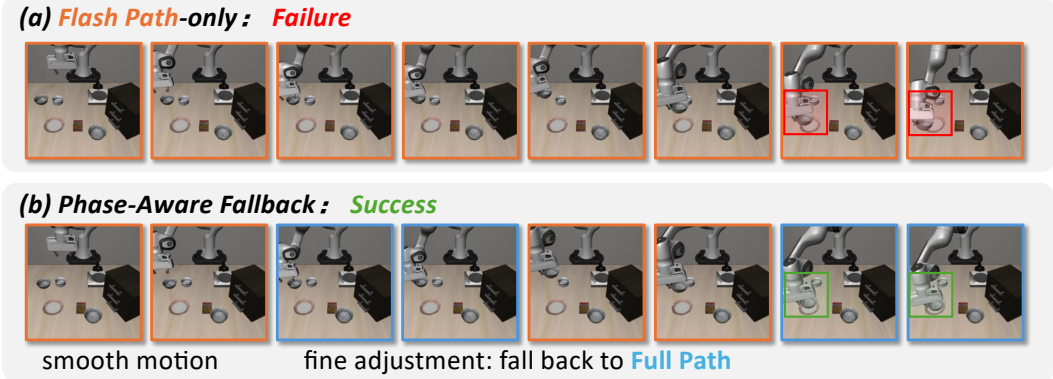


Figure 6: **Key-frame visualization of Figure 5.** Orange borders indicate flash-path execution and blue borders indicate full-path execution after fallback. The failure rollout keeps final placement on the flash path, leaving the bowl misaligned with the plate.

to-plate trajectory without and with phase-aware fallback. Without fallback, flash-path execution continues into the final fine-adjustment phase, where the trajectory drifts toward the plate edge and leaves the bowl misaligned with the plate. Figure 6 provides key frames.

We therefore augment verification-triggered fallback with a phase-aware fallback rule based on the gripper channel, as shown in Figure 5(b). Unlike the continuous position and rotation channels, the gripper channel represents discrete open and closed modes, encoded as -1 and 1 in LIBERO. After applying the same standardization used by the policy, the two modes remain separated around zero, so we detect gripper switches by thresholding the standardized gripper value at zero. Such switches correspond to grasp or release events, which often mark precision-critical fine-adjustment phases.

If a gripper switch appears in any verification branch within the candidate chunk, we treat it as evidence that the rollout has entered a fine-adjustment phase. The system then falls back immediately to the full path to regenerate higher-quality actions, avoiding error accumulation in precision-sensitive phases where small deviations can cause task failure.

4 Experiments

4.1 Setup

We evaluate Realtime-VLA FLASH on π_0 [1] in both simulation and real-world reactive manipulation tasks. All online inference experiments are run on a single NVIDIA RTX 4090D. We use an action chunk size of $H = 50$ and replan after every 12 executed actions.

Simulation. We follow the standard LIBERO setup [17] and report results on four suites: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-10. Each suite contains 10 tasks, and each task is evaluated over 50 trials. We train a separate draft model for each LIBERO suite.

Real-world. For real-world experiments, we use a single-arm UR5 platform equipped with two Intel RealSense D435i cameras, which provide multi-view observations of the workspace. We evaluate conveyor-belt sorting with two object categories: toy dog and hairbrush. For each object category, we collect 200 demonstrations and fine-tune the policy with LoRA for 20k steps. Each real-world condition is evaluated over 10 trials.

4.2 Simulation Evaluation

Setup. We evaluate Realtime-VLA FLASH on LIBERO by comparing four methods: Torch- π_0 [1], Triton- π_0 [19], FLASH- π_0 , and FLASH+Triton- π_0 . We adapt Triton- π_0 to LIBERO and build FLASH+Triton- π_0 by combining our FLASH with the Triton implementation. The structurally aligned draft and verify modules allow partial reuse of existing Triton- π_0 kernels. We report suite-level success rates, average task-level inference latency (**Lat.**), and per-action latency (**/Act**).

Table 1: **Simulation results on four LIBERO suites.** We report suite-level success rates, average success rate (**SR**), average task-level latency (**Lat. (ms)**), average per-action latency (**/Act (ms)**), and relative improvement over the original π_0 [1].

Methods	Success Rate (%)				Average			Improvement	
	Spatial	Object	Goal	10	SR	Lat.	/Act	Δ SR	Speedup
Torch- π_0 [1]	96.8	98.8	95.8	85.2	94.1	58.0	5.0	—	1.00×
Triton- π_0 [19]	96.4	98.8	95.0	86.6	94.2	39.7	3.5	+0.1	1.46×
FLASH- π_0	96.4	99.2	94.6	83.4	93.4	34.9	3.0	-0.7	1.66×
FLASH+Triton-π_0	96.8	99.2	94.4	84.6	93.8	19.1	1.9	-0.3	3.04×

Table 2: **Suite-level flash-path statistics on LIBERO.** **Acc.** is the average accepted draft prefix length in flash-path rounds, normalized by the replan size (12). **Flash Path Rate (FR.)** is the fraction of flash-path rounds among all replanning rounds across task trajectories.

LIBERO Suites	Acc. (%)		FR. (%)		Lat. (ms)		/Act (ms)	
	FLASH	+Tri.	FLASH	+Tri.	FLASH	+Tri.	FLASH	+Tri.
Spatial	75.8	60.2	71.6	68.9	34.3	19.2	3.0	2.1
Object	78.1	89.3	72.4	86.4	34.6	12.6	3.0	1.2
Goal	81.4	66.9	63.7	62.4	37.5	20.5	3.3	2.1
10	82.4	62.5	57.5	49.6	33.2	24.1	2.8	2.3
Avg.	79.4	69.7	66.3	66.8	34.9	19.1	3.0	1.9

Results. Table 1 summarizes the main simulation results. Compared with the 58.0 ms original full-path round, a flash-path round costs only 17.9 ms for FLASH- π_0 and **7.8 ms** for FLASH+Triton- π_0 (Appendix A, Table 6). With phase-aware fallback, FLASH- π_0 reduces the task-level average latency to 34.9 ms, achieving a 1.66× speedup over Torch- π_0 ; with Triton optimization, FLASH+Triton- π_0 further reduces it to **19.1 ms**, achieving a **3.04×** speedup. This also corresponds to a **2.63×** reduction in per-action latency, with only a 0.3-point drop in average success rate.

Table 2 attributes the FLASH speedup to high flash-path usage: FLASH+Triton- π_0 handles 66.8% of replanning rounds via the flash path, with accepted prefixes covering 69.7% of the replan window, thereby reducing full-path calls and amortizing speculative execution. Table 3 evaluates runtime components on LIBERO-10, the most hyperparameter-sensitive suite, where small speculative errors can accumulate and cause task failure. We fix the verification threshold to $\delta = 0.15$ and use two verifier timesteps ($|\mathcal{T}| = 2$), then ablate periodic full-path refresh (**PF**) and phase-aware fallback (**FB**). PF periodically invokes higher-fidelity full-path actions to correct long-horizon drift, while FB performs targeted correction near precision-critical phases indicated by gripper switches. We finally select +FB & PF=2 as the best success-latency trade-off for the LIBERO-10 result in Table 1, with detailed ablations provided in Appendix C.1.

Table 3: **Ablation of key components on LIBERO-10.** PF = n denotes forcing one full-path refresh every n flash-path rounds.

Configuration	SR (%)	Lat. (ms)
Baseline (Flash-path only)	58.4	13.3
<i>Periodic full-path refresh</i>		
+ PF = 2	80.6	21.0
+ PF = 3	82.0	18.6
+ PF = 4	75.4	17.2
<i>Phase-aware fallback</i>		
+ FB	66.8	17.7
+ FB & PF = 2	84.6	24.1
+ FB & PF = 3	81.6	21.9
+ FB & PF = 4	78.4	20.2

4.3 Real-world Evaluation

Setup. We evaluate real-world conveyor-belt sorting, where the robot must grasp a moving object and finish sorting before it leaves the reachable region, as shown in Figure 7. To isolate policy inference latency, all methods use the same synchronous inference loop without asynchronous inference, real-time chunking [2, 3], or other latency-hiding optimizations. We compare JAX- π_0 [1], Triton- π_0 [19], and FLASH+Triton- π_0 under the same hardware and protocol across medium, high, and extra high speeds. Appendix C.2 provides the platform and training hyperparameters.

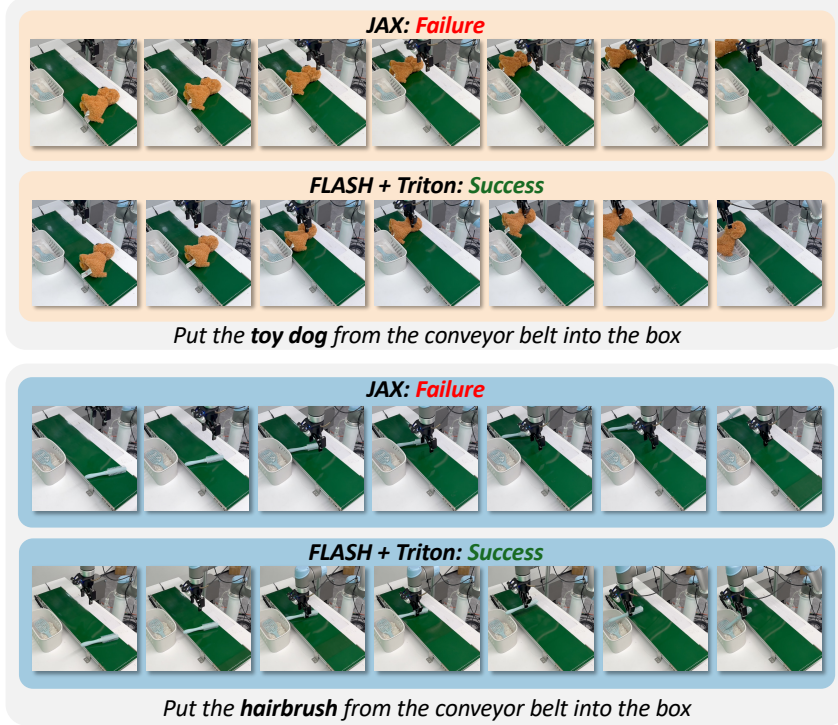


Figure 7: Conveyor-belt sorting demo.

Table 4: **Conveyor-belt sorting results under synchronous inference.** We report success rate (%) over 10 trials for each object and speed; demonstrations are collected at 6 m/min.

Methods	Medium (10 m/min)		High (13 m/min)		Extra High (15 m/min)	
	Toy dog	Hairbrush	Toy dog	Hairbrush	Toy dog	Hairbrush
JAX- π_0 [1]	20.0	50.0	0.0	0.0	0.0	0.0
Triton- π_0 [19]	80.0	40.0	30.0	10.0	0.0	0.0
FLASH+Triton-π_0	80.0	90.0	50.0	30.0	20.0	10.0

Results. Table 4 shows that conveyor speed amplifies the effect of policy inference latency. As a reference point below the reported speed range, JAX- π_0 achieves 80% success on toy dog at 8 m/min, but drops sharply at the reported speeds. At medium speed, all methods achieve nonzero success, but FLASH+Triton- π_0 already improves the more timing-sensitive hairbrush task. As speed increases, the synchronous baselines degrade rapidly: JAX- π_0 [1] fails at high speed, Triton- π_0 [19] retains only limited success, and both fail at extra high speed. In contrast, FLASH+Triton- π_0 remains successful at extra high speed and is the only method with nonzero success at 15 m/min.

Failures of the slower baselines mostly come from stale action chunks: the robot approaches an outdated belt position, so the gripper arrives behind the object or closes too late. This effect is stronger at higher speeds and on hairbrushes, whose thin geometry leaves less tolerance to timing and pose errors. These failures support the main real-world takeaway: under synchronous control, reducing policy latency directly expands the speed range in which dVLAs can complete reactive manipulation.

5 Conclusion

We presented Realtime-VLA FLASH, a speculative inference framework that serves most replanning rounds through the flash path and reverts to the full path when verification or fine-adjustment phases require higher fidelity. Experiments on LIBERO and conveyor-belt sorting show substantial latency reductions while largely preserving task performance, suggesting a practical path toward latency-sensitive embodied control. FLASH still relies on heuristic thresholds, leaving trajectory-adaptive verification thresholds as an important direction for future work.

Acknowledgements

We thank Haoqiang Fan and collaborators for insightful discussions and for the Realtime-VLA line of work [19, 30] which helped shape our focus on latency-aware VLA deployment.

References

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *ArXiv*, abs/2410.24164, 2024.
- [2] Kevin Black, Manuel Y Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025.
- [3] Kevin Black, Allen Z Ren, Michael Equi, and Sergey Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.
- [4] Yang Chen, Xiaoguang Ma, and Bin Zhao. Mean-flow based one-step vision-language-action. *arXiv preprint arXiv:2603.01469*, 2026.
- [5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [6] Valentin De Bortoli, Alexandre Galashov, Arthur Gretton, and Arnaud Doucet. Accelerated diffusion models via speculative sampling. *arXiv preprint arXiv:2501.05370*, 2025.
- [7] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *arXiv preprint arXiv:2410.12557*, 2024.
- [8] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025.
- [9] Hengyuan Hu, Aniket Das, Dorsa Sadigh, and Nima Anari. Diffusion models are secretly exchangeable: Parallelizing ddpms via autospeculation. *arXiv preprint arXiv:2505.03983*, 2025.
- [10] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [11] Wenqi Jiang, Jason Clemons, Karu Sankaralingam, and Christos Kozyrakis. How fast can i run my vla? demystifying vla inference performance with vla-perf. *arXiv preprint arXiv:2602.18397*, 2026.
- [12] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.
- [13] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 7421–7432, 2024.
- [14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.
- [15] Tao Lin, Yilei Zhong, Yuxin Du, Jingjing Zhang, Jiting Liu, Yinxinyu Chen, Encheng Gu, Ziyang Liu, Hongyi Cai, Yanwen Zou, et al. Evo-1: Lightweight vision-language-action model with preserved semantic alignment. *arXiv preprint arXiv:2511.04555*, 2025.

- [16] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [17] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [18] Songming Liu, Bangguo Li, Kai Ma, Lingxuan Wu, Hengkai Tan, Xiao Ouyang, Hang Su, and Jun Zhu. Rdt2: Exploring the scaling limit of umi data towards zero-shot cross-embodiment generalization. *arXiv preprint arXiv:2602.03310*, 2026.
- [19] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running vlas at real-time speed. *arXiv preprint arXiv:2510.26742*, 2025.
- [20] NVIDIA, Johan Bjorck, Nikita Cherniadev, Fernando Castañeda, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. In *ArXiv Preprint*, March 2025.
- [21] NVIDIA GEAR Team, Allison Azzolini, Johan Bjorck, Valts Blukis, et al. Gr00t n1.6: An improved open foundation model for generalist humanoid robots. https://research.nvidia.com/labs/gear/gr00t-n1_6/, December 2025.
- [22] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [23] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [24] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [25] Hanzhen Wang, Jiaming Xu, Yushun Xiang, Jiayi Pan, Yongkang Zhou, Yong-Lu Li, and Guohao Dai. Specprune-vla: Accelerating vision-language-action models via action-aware self-speculative pruning. *arXiv preprint arXiv:2509.05614*, 2025.
- [26] Songsheng Wang, Rucheng Yu, Zhihang Yuan, Chao Yu, Feng Gao, Yu Wang, and Derek F Wong. Spec-vla: speculative decoding for vision-language-action models with relaxed acceptance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 26916–26928, 2025.
- [27] Zhendong Wang, Zhaoshuo Li, Ajay Mandlekar, Zhenjia Xu, Jiaojiao Fan, Yashraj Narang, Linxi Fan, Yuke Zhu, Yogesh Balaji, Mingyuan Zhou, et al. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. *arXiv preprint arXiv:2410.21257*, 2024.
- [28] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- [29] Justin Williams, Kishor Datta Gupta, Roy George, and Mrinmoy Sarkar. Lite vla: Efficient vision-language-action control on cpu-bound edge robots. *arXiv preprint arXiv:2511.05642*, 2025.

- [30] Chen Yang, Yucheng Hu, Yunchao Ma, Yunhuan Yang, Jing Tan, and Haoqiang Fan. Realtime-vla v2: Learning to run vlas fast, smooth, and accurate. *arXiv preprint arXiv:2603.26360*, 2026.
- [31] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. Efficientvla: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*, 2025.
- [32] Jingxuan Zhang, Yunta Hsieh, Zhongwei Wang, Haokun Lin, Xin Wang, Ziqi Wang, Yingtie Lei, and Mi Zhang. Quantvla: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026.
- [33] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [34] Zihao Zheng, Zhihao Mao, Maoliang Li, Jiayu Chen, Xinhao Sun, Zhaobo Zhang, Donggang Cao, Hong Mei, and Xiang Chen. Kerv: Kinematic-rectified speculative decoding for embodied vla models. *arXiv preprint arXiv:2603.01581*, 2026.
- [35] Zihao Zheng, Zhihao Mao, Sicheng Tian, Maoliang Li, Jiayu Chen, Xinhao Sun, Zhaobo Zhang, Xuanzhe Liu, Donggang Cao, Hong Mei, et al. Heisd: Hybrid speculative decoding for embodied vision-language-action models with kinematic awareness. *arXiv preprint arXiv:2603.17573*, 2026.

A Draft Model Details

A.1 Architecture

This section expands the draft architecture introduced in Section 3.3. Because the draft model reuses a VLM block, the observations provide visual context but do not define where a fixed-length action chunk should be written. We therefore append H learned action slots as explicit output positions:

$$Q = [q_1, q_2, \dots, q_H],$$

where q_h corresponds to the h -th future action. These slots are trainable parameters, and they serve as persistent output queries that decode structured future actions from the shared context.

We use a blockwise attention mask to make these slots compatible with the reused VLM architecture. Each action slot attends to the visual-language prefix and state token, and the slots can also interact within the action block, allowing the predicted actions to be coordinated across the chunk. Because all H slots are processed in the same forward pass, the transformer produces hidden states for the entire action chunk in parallel.

After the transformer block, only the hidden states associated with the action slots are decoded. A shared linear action head maps each slot representation to its corresponding continuous action:

$$\hat{a}_{t+h-1}^d = W_{\text{act}} z_h + b_{\text{act}}, \quad h = 1, \dots, H,$$

where z_h is the final hidden representation of slot q_h .

Thus, the learned action slots convert a VLM block into a parallel action-chunk predictor.

A.2 Training Hyperparameters and Inference Cost Breakdown

Table 5 summarizes the draft-model training setup. The draft model regresses to frozen full-path policy outputs with a prefix-weighted objective, so early executable actions receive higher weight while verification and fallback still decide the final executed prefix.

Table 6 reports the component-level latency behind the task-level results in Table 1. We include both per-round component costs and trajectory-level averages because flash-path rounds may execute variable-length prefixes.

Table 5: Training hyperparameters for the draft model.

Item	Value
Optimizer	AdamW
Learning rate	2×10^{-3}
Weight decay	0.01
Batch size	64
Epochs	100
Precision	FP32
Loss	Weighted Smooth L1 / Huber loss, $\beta = 1.0$
Prefix weighting	Sampled prefix up to 16 steps, $\gamma_{\text{prefix}} = 0.9$, tail weight 0.1
Checkpoint	Best validation RMS over the first 12 executable steps
Compute cost	4 NVIDIA RTX 4090D GPUs, approximately 6 hours per draft model

Table 6: Inference cost breakdown.

Component	Torch- π_0	Triton- π_0	FLASH- π_0	FLASH+Triton- π_0
<i>Full-path round (ms)</i>				
Image encoder	11.3	4.7	11.3	4.7
VLM prefill	26.7	22.4	26.7	22.4
Action denoise	20.0	12.6	20.0	12.6
Full-path total	58.0	39.7	58.0	39.7
<i>Flash-path round (ms)</i>				
Image encoder	–	–	11.0	4.7
Draft model	–	–	3.5	0.9
Parallel verifier	–	–	3.4	2.2
Flash-path total	–	–	17.9	7.8
<i>Trajectory-level average</i>				
Replanning latency	58.0	39.7	34.9	19.1
Per-action latency	5.0	3.5	3.0	1.9
Speedup	1.00×	1.46×	1.66×	3.04×

B Verification Consistency Interpretation

The endpoint check in Algorithm 1 should be interpreted as a heuristic local consistency test, rather than as a formal guarantee that the accepted draft and the full-path rollout induce identical trajectories or task-level behavior. The key distinction is that the Action Expert is trained by flow matching on interpolation paths toward target action endpoints, whereas verification probes the learned flow field on states induced by a drafted endpoint. Thus, passing verification indicates that the draft is locally compatible with the learned velocity field under the reused conditioning context, but it does not establish full-path equivalence.

Let A_t^* denote the endpoint produced by a full-path rollout under the current conditioning context, and let $\hat{A}_t^{(d)}$ denote the accepted draft endpoint. Informally, the endpoint discrepancy can be viewed as controlled by the acceptance threshold, the local reconstruction error of the Action Expert, the residual conditioning mismatch, and the mismatch between target-induced and draft-induced interpolation paths:

$$\|\hat{A}_t^{(d)} - A_t^*\| \lesssim \delta + \epsilon_{\text{AE}} + \epsilon_{\text{cond}} + \epsilon_{\text{path}}.$$

Here, ϵ_{AE} accounts for approximation errors in the local velocity field and the single-step endpoint reconstruction performed by the Action Expert. The term ϵ_{cond} captures residual mismatch introduced by reusing the cached visual-language prefix. The term ϵ_{path} captures the fact that verification evaluates the flow field on draft-induced interpolation states, rather than on the target-endpoint paths used during training or the states visited by a full sequential rollout.

Importantly, as shown in Algorithm 1, the robot state s_t is refreshed at every verification call and therefore does not constitute a stale cached variable. Overall, the acceptance rule should be understood as a conservative local agreement criterion, not as a formal correctness guarantee for the accepted prefix. This interpretation motivates conservative thresholds, multi-timestep verification, phase-aware fallback, and full-path fallback when local consistency fails.

C Experimental Details

C.1 Simulation Details: Verification Hyperparameter Ablation

We isolate the verifier before adding phase-aware fallback and periodic full-path refresh, so this subsection should be interpreted as a verifier trade-off study. Table 7 uses LIBERO-10 as a stress test and varies the number of verification timesteps $K = |\mathcal{T}|$ and distance threshold δ . Loose thresholds keep more rounds on the flash path and reduce latency, but they also allow inaccurate drafts to pass verification. Stricter thresholds and larger K reject more speculative actions, improving reliability in sensitive settings while pushing latency toward the full-path regime.

We use $K = 2, \delta = 0.15$ as the base verifier setting for the full runtime rather than as a standalone verifier-only optimum. This point preserves high flash-path coverage and low latency, while the complete runtime recovers reliability through phase-aware fallback and periodic full-path refresh (Table 3). Table 8 further fixes $K = 2$ and sweeps δ across all LIBERO suites. The same acceptance-latency pattern appears across suites, while LIBERO-10 shows the largest success-rate variation; we therefore use it as the primary verifier stress test in Table 7.

Table 7: Verifier-only ablation on LIBERO-10.

Verifier Setting			LIBERO-10 Result				
K	Lat. (ms)	δ	SR (%)	Lat. (ms)	/Act (ms)	Acc. (%)	FR. (%)
4	4.3	0.20	82.6	35.7	4.2	13.9	18.8
		0.15	86.2	39.9	4.4	6.8	9.2
		0.10	85.0	47.5	4.5	1.8	2.1
		0.05	84.4	52.4	4.5	0.0	0.0
		0.00	86.4	52.3	4.5	0.0	0.0
2	2.2	0.20	54.3	10.3	0.9	91.8	94.3
		0.15	58.4	13.3	1.5	65.6	83.0
		0.10	79.4	21.1	3.3	23.3	40.9
		0.05	93.5	44.9	4.5	2.0	2.5
		0.00	85.6	47.9	4.1	0.0	0.0
1	1.4	0.20	54.9	13.5	1.2	99.7	95.4
		0.15	53.8	12.5	1.1	99.4	95.4
		0.10	51.0	12.9	1.2	84.8	92.4
		0.05	84.2	28.5	4.0	11.4	19.6
		0.00	86.8	49.8	4.2	0.0	0.0

Table 8: Cross-suite threshold sensitivity with $K = 2$.

δ	Success Rate (%)				Latency (ms)				Average				
	Spatial	Object	Goal	10	Spatial	Object	Goal	10	SR	Lat.	/Act	Acc.	FR.
0.30	95.8	99.2	86.6	53.8	12.0	11.1	11.8	9.9	83.9	11.1	1.0	99.7	91.3
0.25	93.6	98.4	87.6	53.8	11.9	11.3	11.8	10.0	83.4	11.1	1.0	98.8	91.4
0.20	94.8	98.6	86.4	54.3	11.8	11.3	11.9	10.3	83.5	11.2	1.0	95.1	90.9
0.15	96.0	99.2	90.8	58.4	12.3	12.6	12.5	13.3	86.1	12.7	1.4	77.5	84.9
0.10	97.2	98.8	92.6	79.4	19.5	20.7	19.5	21.1	92.0	20.4	3.0	30.5	46.6
0.05	93.8	98.8	94.4	87.2	41.0	40.2	40.9	41.6	93.5	41.0	4.1	2.0	2.5
0.00	97.0	97.6	96.2	85.6	47.4	47.2	47.2	47.9	94.1	47.5	4.1	0.0	0.0

C.2 Real-world Details

Figure 8 illustrates the real-world evaluation platform used for conveyor-belt sorting. In the real-world deployment, accepted actions are still executed through the robot controller with joint limits and command constraints, so a false acceptance remains subject to these low-level safety constraints.

Table 9 reports the training configuration. We first fine-tune the main policy from the pre-trained π_0 checkpoint using LoRA on the collected demonstrations, and then train the draft model for the same real-world domain. For draft training, the observations come from the real demonstrations, while the regression targets are teacher action chunks generated by the fine-tuned main model rather than directly by behavior cloning from raw demonstration actions.

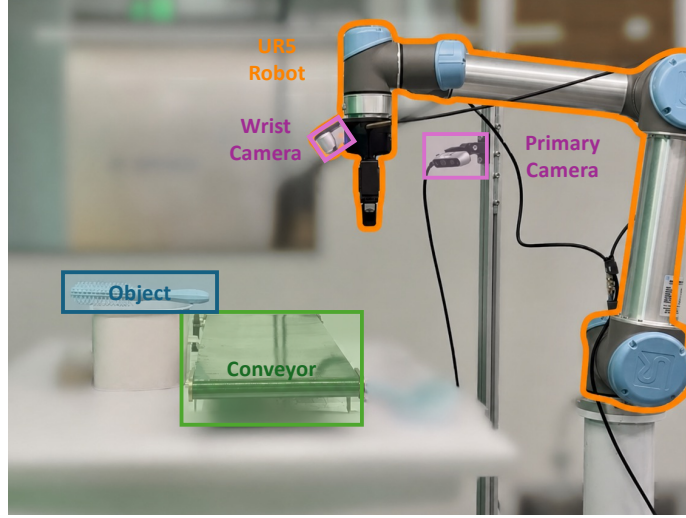


Figure 8: Real-world robot platform used for conveyor-belt sorting.

Table 9: Training hyperparameters for real-world demonstration adaptation.

Item	Value
<i>Main Model fine-tuning</i>	
Batch size	128
Training steps	20k
LR schedule	Cosine decay with 500 warmup steps
Peak / final LR	$1 \times 10^{-4} / 1 \times 10^{-5}$
<i>Draft model training</i>	
Optimizer	AdamW
Learning rate	2×10^{-3}
LR schedule	Cosine schedule with 1 warmup epoch and 1×10^{-5} minimum LR
Batch size	64 per GPU
Epochs	100
Gradient clipping	Global norm 1.0
Loss	Weighted Smooth L1 / Huber loss, $\beta = 1.0$
Prefix weighting	Sampled prefix up to 20 steps, $\gamma_{\text{prefix}} = 0.9$, tail weight 0.1
Checkpoint	Best validation RMS over the first 12 executable steps
Compute cost	8 NVIDIA H20 GPUs, approximately 12 hours for fine-tuning and 2 hours for draft model training

D Future Work

Realtime-VLA FLASH suggests several directions for future work.

First, the current verifier uses heuristic hyperparameters, including fixed acceptance thresholds and hand-picked verification timesteps. A more adaptive verifier could adjust both the threshold and the timestep locations over the course of a trajectory, tightening verification during fine-adjustment phases, rapid observation changes, or high-curvature motions, while relaxing it during smooth free-space motion. This generalizes the intuition behind our phase-aware fallback mechanism from a gripper-switch heuristic to a broader context-dependent verification rule.

Second, FLASH could be combined with real-time chunking (RTC) methods. Current RTC methods often assume a fixed inference latency, whereas FLASH introduces dynamic accepted prefix lengths and variable effective latency across replanning rounds. Adapting RTC to this setting would require trajectory optimization algorithms that account for variable-latency policy updates, potentially yielding trajectories that are both fast and smooth.

Third, FLASH may further benefit edge deployments. VLA-Perf [11] reports that VLA inference stages become memory-bound on edge devices, where memory bandwidth, power, and thermal budgets are substantially tighter than on desktop GPUs. By reducing repeated full-path inference calls, FLASH can lower average memory traffic and power consumption during replanning. Extending this design to resource-constrained edge devices could make low-latency dVLA deployment more practical for robots with limited compute, power, or thermal budgets.